斩获冠军！清华 x 腾讯混元登顶 MLSys 2026 MoE 模型推理优化竞赛

腾讯技术工程 May 23, 2026, 9:00 PM 广东 38 listened



随着大模型走向万亿参数时代，MoE（混合专家）架构成为主流，但其在异构芯片（NPU）上的推理性能还面临着诸多挑战。近日，在国际顶级机器学习系统会议 MLSys 2026 的MoE推理优化挑战赛中，清华大学与腾讯混元AI infra团队从全球众多顶尖团队中脱颖而出，成功夺冠。团队针对 MoE 模型，设计全链路优化方法，将在官方指定 NPU 上推理速度提升 4.1 倍，彰显了团队在大模型推理系统领域的深刻积累

一、赛事和任务介绍

MLSys 是聚焦机器学习系统领域的国际顶级会议，议题覆盖训练、推理、编译器等AI infra核心话题。MLSys 2026 设置了专项的 MoE 优化挑战赛。根据赛事官方要求，参赛队伍需要在指定NPU上为指定模型的关键模块编写自定义算子，并在保证正确性的前提下优化整个解码推理流程的端到端延迟。

来自清华存储实验室的队员，从数据搬运与片上存储利用的视角出发，解决MoE推理过程中DMA带宽利用不充分、激活在HBM与片上缓存之间反复搬运等关键性能瓶颈；腾讯混元AI Infra团队提供大模型推理与算子上的丰富经验与深度指导，团队重点优化了权重加载流水、片上数据布局与模块间数据复用策略，最终实现了端到端解码延迟的降低4.1倍，以显著优势赢得冠军。

本次赛事竞争十分激烈，Stanford、MIT、UCBerkeley、CMU、UCLA、Cornell 等多所国际知名高校均进入第二阶段角逐。团队最终脱颖而出，充分展现了其在大模型推理系统、异构芯片适配和底层算子优化方面的综合实力。

冠军团队参赛成员为高世伟、范如文、任庭旭、罗以彬，指导者包括清华舒继武教授、陆游游副教授、腾讯AIInfra专家reed，也得到腾讯混元专项博士生科研激励计划的大力支持。

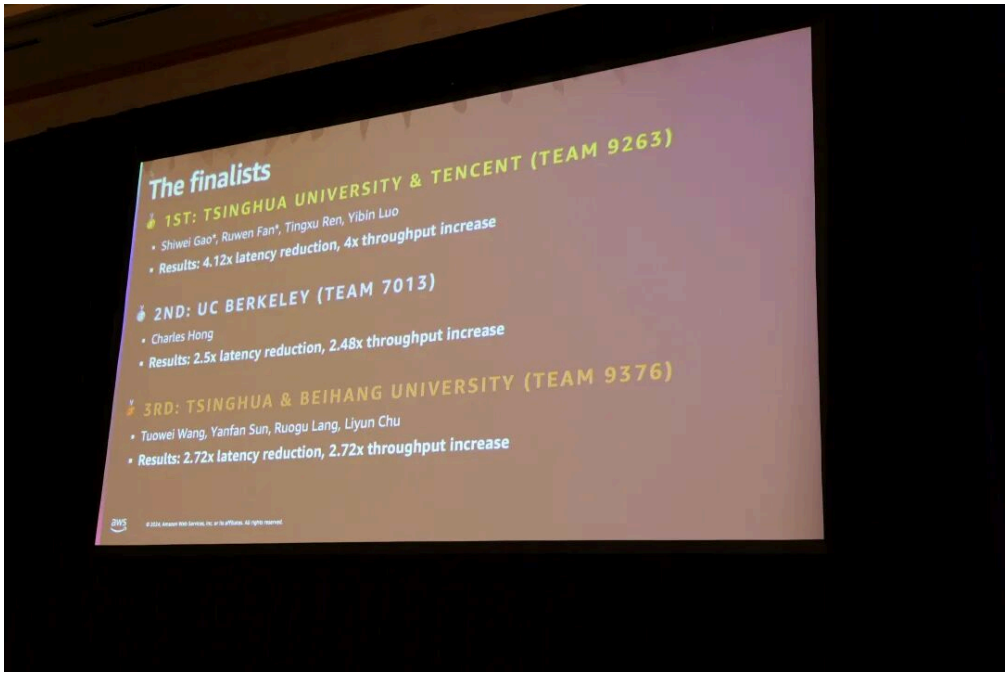


图1：大赛最终获奖结果



图2：团队成员在MLSys大会上报告竞赛解决方案

比赛链接：https://github.com/aws-neuron/nki-moe

二、 性能优化

团队首先在NPU上对现有推理系统进行了性能分析。结果显示，MoE 模块单层耗时约0.2 ms，占单步生成延迟的 82% 以上，是整个推理链路中的主要瓶颈。模块的主要耗时来自权重加载和中间数据搬运。计算时，数据需要从大容量的HBM中加载到片上缓存，才能被计算单元使用与计算。团队分析并发现，由于当前系统中的指令顺序和数据排布对硬件不友好，数据传输引擎的 DMA 带宽没有被充分利用，严重拖慢了MoE模块的计算速度。此外，注意力模块的单层耗时约为 40 us，也是系统优化的关键之一。该模块涉及 QKV、RoPE、KV 缓存更新之间的频繁的数据

布局转换和中间搬运，同样存在大量优化空间。为此，团队围绕上述两大核心模块的访存与计算特点，打出了一套优化“组合拳”，最终大幅提高了系统的整体执行效率。

MoE：提升权重搬运效率

1. 采用 E-Shard 策略，按专家而不是按隐藏维度拆分双核任务。基线实现采用张量并行，每个计算核心（LNC）分得的隐藏维度只有 96，导致小块DMA开销超过并行收益。团队改为在计算核心间上按专家切分任务并行，LNC0 处理前 4 个专家，LNC1 处理后 4 个专家。每个核心独立完成对应专家的完整计算，减少跨核同步。采用E-Shard后，Gate和Up矩阵能用一次连续 DMA 同时搬入 SBUF，避免原实现中权重分片加载导致的低效访存。

2. 使用 PSUM 批量读出，减少片上结果搬运指令数。 原始路径会为每个 PSUM （矩阵核心私有寄存器）tile单独发起一次拷贝，tile 数越多，读出指令越多。每次拷贝都会带来大量的启动开销，导致资源浪费。团队将多个 PSUM 存储区组织在一个连续的三维张量中，每个存储区通过切片视图参与矩阵乘累加，读出时再用一次批量拷贝将多个存储区一起搬出。这样可以把多次分散读出合并为一次批量读出，减少指令发射开销。

3. 针对解码场景设计专门 GEMV 路径，充分利用所有 PSUM 并发访问能力。为了针对性优化推理延迟，比赛使用批大小为一的输入作为测试用例，此时很多矩阵乘退化成 GEMV，单个计算往往只占用一个 PSUM Bank。由于PSUM寄存器单写限制，片上并发能力没有被充分释放。团队在 PSUM 批量读出的基础上，将 GEMV 输出打散到多个 PSUM Bank中并发执行，提高计算并行度。

4. 用标量引擎 DMA 处理首个专家，降低初始搬运等待。 在专家权重加载开始阶段，第一次 DMA 的启动延迟会直接影响后续流水。团队利用标量引擎 DMA 指令派发延迟更低的特点，提前启动第一个专家权重的DMA；后续再切换到常规的高吞吐 DMA 路径。这个设计降低DMA启动阶段的空泡，让数据搬运更快进入稳定状态。

5. 抑制编译器自动预取，避免与关键路径争抢 DMA 带宽。 默认情况下，编译器会把注意力权重放在堆空间尾部（从大到小编址），把 MoE 权重放在栈空间（从小到大编址）。注意力计算结束后，堆空间尾部被释放，编译器立即使用这块空间预取下一层权重。权重预取会与残差加法和集合通信争抢 DMA 带宽，反而拉长端到端延迟。团队将 MoE 权重也放到堆空间中，占住堆空间尾部，从而阻止编译器在错误时机发起自动预取，把 DMA 带宽留给关键的高优路径。



图3：MoE 优化结构示意图，包含E-Shard 专家切分、连续 DMA、PSUM/GEMV 并发、冷启动流水和预取控制。

Attention：减少中间数据搬运和布局转换

针对模型中的注意力模块，团队将 QKV 投影、注意力计算和输出投影视为一个整体，通过重新设计片上数据布局和双核切分方式，减少 HBM 往返和模块边界开销。

1. QKV 投影直接生成后续计算所需布局，减少转置和 HBM 中转。原始路径会先生成通用 [B, S * I] 布局，再经过切分、转置、写 HBM、读 HBM 后进入注意力计算。优化后，Q/K/V 在投影阶段直接写入适合后续计算的 [d_head, B * S_tkg] 短搬运布局。这样 Q 和 K 可以直接进入 RoPE 和注意力计算，无需额外转置；V 仅在注意力模块内部保留一次 PSUM 转置，用于后续 Score * V 和 KV 缓存更新。

2. 让 V 常驻片上内存，省掉一次完整 HBM 往返。基线实现中，V 会先写回 HBM，再由注意力模块重新读入，带来额外带宽消耗和同步等待。优化后，V 在 QKV 投影后继续保留在 SBUF 中，并直接传给注意力计算；KV 缓存更新也复用同一份片上缓冲区。若位置编号已经位于 SBUF，掩码生成也可以直接复用片上数据，进一步避免额外 HBM 读取。

3. 按 d_head 维度切分 KV 缓存更新，改善双核负载均衡。原始 KV 缓存更新容易出现双核工作量不均，一侧等待另一侧完成的情况。团队改为按 d_head 维度切分任务，每个 LNC 负责写入自己对应的头维度分片，减少跨核等待和空转，提高 KV 更新阶段的并行效率。

4. 融合 Attention、残差加法和 MoE，减少模块边界写回。在上述优化基础上，团队进一步将一层 Transformer 中的 Attention -> 残差加法 -> MoE -> 残差加法 融合到同一个算子中。融合后，中间结果不再需要在每个模块边界写回 HBM，残差加法也可以直接在 SBUF 中完成。

三、精度对齐

本次竞赛不是只看速度。评测要求，输出结果和基线需要比特级对齐，这意味着优化后的算子不仅要高性能，输出还必须尽可能和基线的累加行为完全一致。对推理优化来说，这比通常意义上的“数值接近”要苛刻得多。

难点在于，上部分介绍的高性能算子实现会改变计算路径与顺序。团队采用的 E-Shard 会让两个核心各自完成 4 个专家的完整计算，而基线的 H-Shard 则沿中间维度拆分。两种策略的浮点累加顺序不同，即使数学公式完全等价，也会因为舍入顺序不同产生误差。

团队通过近一周时间的反复调试定位，逐步找到关键差异。最核心的问题出现在下投影阶段：基线在处理多个专家的FP32输出时，会使用BF16逐个累加所有专家的输出（每个专家有一次截断）。这个“加一次、截断一次”的过程会从第 1 个专家一直重复到第 8 个专家，导致误差累积。而E-Shard实现中，累加方法为每个计算核上的4个专家先以FP32高精度累加，最后两个核心结果相加到BF16结果，仅截断一次。

团队在 E-Shard 架构中复现了基线的行为：先通过核间通信机制在两个核心之间汇集专家结果，再严格按照基线的逐专家顺序执行 FP32 到BF16 的累加。此外，RMSNorm、路由计算、门控/上投影、激活和中间数据类型转换也逐项与基线对齐。这样既保证了kernel的高性能，也保证了输出的比特级对齐。

四、One More Thing— Knight: Agent算子优化器

优异成绩背后，离不开联合团队共同设计研发的一套基于Agent的推理算子优化器：Knight。在激烈的比赛中，参赛团队晚上安排Knight针对算子进行自动化优化，白天根据系统反馈进行手工调优、指导Knight的下一步优化方向，实现“人停卡不停”的效果。

Knight的主要流程包括提出假设、实现修改、远程验证、失败归因和经验沉淀，组织成一个可持续迭代的循环。它包含方案提出者、实现者、复盘者三个Agent：方案提出者根据历史尝试和已验证经验提出算子优化假设，明确修改计划、验证方式、潜在风险和预期收益；实现者负责落地代码修改，并在真实硬件上反复编译、运行和bug fix；复盘者则在一轮成功/失败后复盘完整材料，包括优化方案、迭代日志、远程输出、最终代码差异和性能指标，把有效经验写入记忆，供下一轮继续使用。

Knight通过记忆系统维护多轮优化的经验。系统用 SQLite 持久化保存尝试记录、经验记录、执行记录和任务记录；每轮开始前检索已验证经验和历史尝试，形成记忆索引；提示词中只注入必要的索引信息，细节按需读取，避免上下文过长；每轮结束后再刷新提示词上下文，让下一轮优化可以继承最新经验。

为了提升自动优化的可靠性，并避免 reward hacking，Knight 加入了若干约束机制。实现者提出的修改都需要在硬件上同时通过正确性和性能验证：正确性侧检查绝对误差、相对误差等指标，性能侧从 profile 日志中解析延迟并计算加速比。系统还会对失败日志进行分类，区分编译失败、测试崩溃、远程环境异常和指标缺失，帮助实现者快速定位原因。与此同时，路径范围检查器通过白名单和黑名单限制可修改文件，防止智能体修改评测脚本或绕过验证。

为了增强通用Agent对异构NPU硬件领域专用知识的了解，团队还建设了 Skills库和扩展实验平台。知识库目前包括NPU上的离线编译、性能剖析、访问模式分析、并行模型分析等常用能力，覆盖张量视图布局、SBUF硬件分区、DMA模式、张量/向量引擎并发模型等坑点。实验平台则扩展出仅编译、编译基线、仅分片、基线输出分数对比、逐词元差异调试和性能分解等功能，让每个候选方案都能更快、更稳定地被验证。

在这次比赛中，Knight帮助团队发现了多个人类程序员难发现的优化点。这套方法为后续 NPU 上的大模型推理优化提供了新的工程路径：在专家经验的引导下，让自动化系统不断扩展优化搜索空间并提升性能。

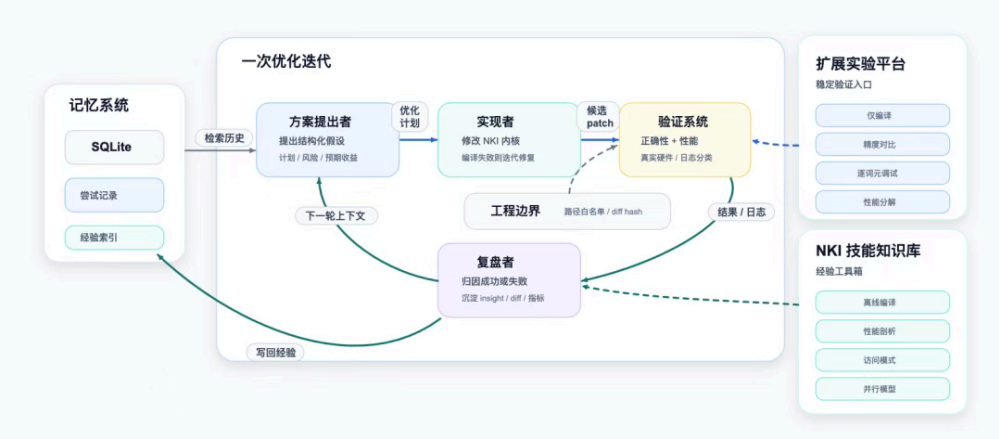


图4：Knight Optimizer 架构关系图。

五、结果展示

通过综合上述优化，团队将比赛指定模型的推理的端到端时间从14.91s降到3.56s，提升约4.1倍。针对单步decode解码，团队将延迟从 12.63 ms 降至 5.45 ms，进一步拆解到核心算子 Fused MoE，单层 MoE 从原始实现的约 204 us 降至约 53 us。团队重构数据搬运路径后，权重加载期间的 DMA 引擎利用率提升到约 80%。

项目	优化前或对照	优化后
端到端推理时间	14.91s	3.56s
完整单步解码前向计算	12.63 ms	5.45 ms
单层 MoE	204 us	53 us
MoE 权重加载期间 DMA 利用率	较低	约 80%

表 1：主要性能结果

从性能剖析结果看，MoE 是原始解码链路中的主要瓶颈。原始实现中，单层 MoE 延迟约为 204 us，DMA 引擎利用率较低，计算单元卡在等待权重和中间数据完成搬运。



图 5：原始实现中，MoE 单层延迟约 204 us，DMA 引擎利用率较低。

在团队重写 MoE 算子并优化数据搬运路径后，单层 MoE 延迟进一步下降到约 53 us，权重加载期间的 DMA 引擎利用率提升到约 80%。这一结果说明，围绕专家切分、连续 DMA、PSUM 并发和预取控制的优化能够有效转化为真实的推理收益。



图 6：优化后，MoE 单层延迟约 53 us，权重加载期间 DMA 引擎利用率提升到约 80%。

六、展望

MoE 正在成为万亿参数时代的大模型主流结构。随着模型参数规模继续增长，MoE 推理更具挑战，需同时处理专家调度、跨芯片通信、片上缓存和专家计算之间的整体协同。未来，团队将进一步开展面向异构硬件开展大模型推理系统优化，尤其关注在以超节点为代表的新型算力平台上的大模型高效部署。

同时，团队也将继续改进基于 Agent 的算子自动优化系统，进一步开发结合硬件知识库、编译器反馈、profile 分析和自动验证平台，形成面向多种后端的持续优化能力。

参考链接

[1] MLSys 2026：https://mlsys.org/





阅读 8766

留言

写留言

还没有留言，来写首评